

Product Review and Rating Management System

DBMS / MySQL Task

Task Objectives

1. Create Review and Rating tables.
2. Store customer feedback and ratings.
3. Retrieve product review details.
4. Calculate average product ratings using aggregate functions.
5. Identify highly rated products.

Complete MySQL Code

```
-- PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

CREATE DATABASE ProductReviewDB;
USE ProductReviewDB;

-- 1. CREATE PRODUCT TABLE
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    Price DECIMAL(10,2)
);

-- 2. CREATE CUSTOMER TABLE
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    Email VARCHAR(100)
);

-- 3. CREATE REVIEW TABLE
CREATE TABLE Review (
    ReviewID INT PRIMARY KEY,
    ProductID INT,
    CustomerID INT,
    ReviewText VARCHAR(500),
    ReviewDate DATE,
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

-- 4. CREATE RATING TABLE
CREATE TABLE Rating (
    RatingID INT PRIMARY KEY,
    ReviewID INT,
    RatingValue INT,
    FOREIGN KEY (ReviewID) REFERENCES Review(ReviewID),
    CHECK (RatingValue BETWEEN 1 AND 5)
);

-- 5. INSERT PRODUCT DATA
INSERT INTO Product VALUES
(101, 'Laptop', 'Electronics', 55000.00),
(102, 'Smartphone', 'Electronics', 25000.00),
(103, 'Headphones', 'Accessories', 3000.00),
(104, 'Smart Watch', 'Accessories', 5000.00),
(105, 'Keyboard', 'Computer Accessories', 1500.00);

-- 6. INSERT CUSTOMER DATA
INSERT INTO Customer VALUES
(1, 'Arun', 'arun@gmail.com'),
(2, 'Priya', 'priya@gmail.com'),
(3, 'Kavin', 'kavin@gmail.com'),
(4, 'Divya', 'divya@gmail.com'),
(5, 'Rahul', 'rahul@gmail.com');

-- 7. INSERT REVIEW DATA
INSERT INTO Review VALUES
(201, 101, 1, 'Very good laptop with excellent performance', '2026-09-01'),
```

```
(202, 101, 2, 'Good laptop for students', '2026-09-02'),
(203, 102, 3, 'Excellent smartphone and good camera', '2026-09-03'),
(204, 102, 4, 'Battery backup is very good', '2026-09-04'),
(205, 103, 5, 'Sound quality is excellent', '2026-09-05'),
(206, 104, 1, 'Good features and attractive design', '2026-09-06'),
(207, 105, 2, 'Keyboard quality is good', '2026-09-07');
```

```
-- 8. INSERT RATING DATA
INSERT INTO Rating VALUES
```

```
(301, 201, 5),
(302, 202, 4),
(303, 203, 5),
(304, 204, 4),
(305, 205, 5),
(306, 206, 4),
(307, 207, 3);
```

```
-- 9. RETRIEVE PRODUCT REVIEW DETAILS
SELECT
```

```
    p.ProductName,
    c.CustomerName,
    r.ReviewText,
    r.ReviewDate,
    rt.RatingValue
FROM Product p
JOIN Review r ON p.ProductID = r.ProductID
JOIN Customer c ON r.CustomerID = c.CustomerID
JOIN Rating rt ON r.ReviewID = rt.ReviewID;
```

```
-- 10. CALCULATE AVERAGE PRODUCT RATING
SELECT
```

```
    p.ProductName,
    AVG(rt.RatingValue) AS AverageRating
FROM Product p
JOIN Review r ON p.ProductID = r.ProductID
JOIN Rating rt ON r.ReviewID = rt.ReviewID
GROUP BY p.ProductID, p.ProductName;
```

```
-- 11. IDENTIFY HIGHLY RATED PRODUCTS
SELECT
```

```
    p.ProductName,
    AVG(rt.RatingValue) AS AverageRating
FROM Product p
JOIN Review r ON p.ProductID = r.ProductID
JOIN Rating rt ON r.ReviewID = rt.ReviewID
GROUP BY p.ProductID, p.ProductName
HAVING AVG(rt.RatingValue) >= 4;
```

```
-- 12. DISPLAY PRODUCTS FROM HIGHEST TO LOWEST RATING
SELECT
```

```
    p.ProductName,
    AVG(rt.RatingValue) AS AverageRating
FROM Product p
JOIN Review r ON p.ProductID = r.ProductID
JOIN Rating rt ON r.ReviewID = rt.ReviewID
GROUP BY p.ProductID, p.ProductName
ORDER BY AverageRating DESC;
```